1. Inputs de Texto

1.1. [TextField]

Menção honrosa. É o widget base para entrada de texto no Flutter.

- **Uso:** Buscas simples, chats, ou inputs isolados que não requerem validação complexa ou integração com um estado de formulário.
- **Nota:** Geralmente evitamos em formulários de cadastro pois ele não possui integração nativa com o widget `Form` (não tem `validator` nem `onSaved` nativos da mesma forma que o `TextFormField`).

1.2. [TextFormField]

É o input mais utilizado para cadastros. Basicamente é um `TextField` envelopado ("Wrapper") que se integra automaticamente com o estado de um `Form`.

1.2.a. Estilização (decoration)

A propriedade `decoration` recebe um `InputDecoration`, onde configuramos o visual e a UX do campo:

- `label`: Texto que flutua. Fica sobre o input quando vazio e "sobe" para a borda quando o usuário clica ou preenche.
- `hintText`: O "placeholder" (texto cinza de ajuda) que aparece apenas quando o campo está vazio. Diferente do label, ele não sobe, ele some.
- `border` / `enabledBorder` / `focusedBorder`: Controle das linhas, cores e arredondamento das bordas em cada estado.
- `prefixIcon` / `suffixIcon`: Ícones dentro da caixa de texto (ex: ícone de e-mail no início ou o "olho" de senha no final).

1.2.b. Tipos de Teclado (keyboardType)

Define qual layout de teclado o sistema operacional deve abrir. Isso é crucial para a UX (não obrigue o usuário a abrir o teclado de texto para digitar um número).

```
TextInputType.text // (Padrão) Teclado normal completo
TextInputType.emailAddress // Adiciona o @ e o . em destaque
TextInputType.number // Teclado numérico (pode conter pontuação dependendo
do OS)
TextInputType.phone // Teclado numérico específico para discagem
TextInputType.datetime // Teclado para datas/horas
TextInputType.multiline // Permite quebrar linhas (usar em conjunto com
maxLines: null)
```

Exemplo de uso:

```
TextFormField(
  keyboardType: TextInputType.emailAddress,
```

```
);
```

1.2.c. Máscaras e Filtros (inputFormatters)

É aqui que impedimos o usuário de digitar letras em um campo de CPF ou formatamos o texto enquanto ele digita (R$ 10,00).

1. Filtros Nativos: O Flutter já traz filtros básicos.

```
inputFormatters: [
  FilteringTextInputFormatter.digitsOnly, // Só aceita números
  FilteringTextInputFormatter.allow(RegExp(r'[a-zA-Z]')), // Só letras
],
```

2. Máscaras Complexas (Pacotes Externos): Para formatos brasileiros (CPF, CNPJ, Telefone, Real), a comunidade utiliza muito o pacote `brasil_fields` ou `mask_text_input_formatter`. O pacote `brasil_fields` já entrega formatadores prontos que aplicam a máscara visual e validam o formato.

```
// Exemplo usando o pacote brasil_fields:
TextFormField(
  keyboardType: TextInputType.number, // Importante mudar o teclado
também!
  inputFormatters: [
    FilteringTextInputFormatter.digitsOnly,
    CpfInputFormatter(), // Formata automaticamente: 000.000.000-00
  ],
);
```

1.2.d. Ação do Teclado (textInputAction)

Altera o botão de "Enter" no canto inferior direito do teclado. Melhora o fluxo de preenchimento.

TextInputAction.done // (Padrão) Ícone de "Check". Fecha o teclado. TextInputAction.next // Ícone de "Seta". Move o foco para o próximo input. TextInputAction.search // Ícone de "Lupa". Geralmente dispara uma busca. TextInputAction.send // Ícone de "Avião". Comum em chats.

Exemplo de fluxo (Campo Nome -> Campo Email -> Enviar):

```
// Campo 1
TextFormField(
  textInputAction: TextInputAction.next,
),
// Campo 2 (Último)
TextFormField(
  textInputAction: TextInputAction.done,
```

```
  onFieldSubmitted: (_) => funcaoDeEnviar(), // Dispara ao clicar no botão
),
```

1.2.e. Senhas e Segurança (obscureText)

Oculta os caracteres digitados. Essencial para senhas.

```
bool _showPassword = false;

TextFormField(
  obscureText: !_showPassword, // Se true, mostra bolinhas. Se false,
mostra texto.
  decoration: InputDecoration(
    suffixIcon: IconButton(
      icon: Icon(_showPassword ? Icons.visibility : Icons.visibility_off),
      onPressed: () {
        setState(() {
           _showPassword = !_showPassword;
        });
      },
    ),
  ),
);
```

1.2.f. Limites e Dimensões (maxLength & maxLines)

1. maxLength: Define um limite rígido de caracteres e mostra um contador nativo no canto inferior direito (ex: 0/140).

```
TextFormField(
  maxLength: 140,
  // buildCounter: ... // Pode customizar ou esconder o contador se quiser
);
```

2. maxLines: Define a altura do input.

- Se for `1` (padrão), o texto corre para a lateral.
- Se for `null`, o input cresce infinitamente conforme o usuário digita (estilo WhatsApp).
- Se for um número fixo (ex: `5`), ele já nasce com a altura de 5 linhas.

```
// Exemplo de campo de "Observação" ou "Bio":
TextFormField(
  keyboardType: TextInputType.multiline,
  maxLines: null, // Cresce automático
  minLines: 3,    // Começa com altura de 3 linhas
);
```

---

2. Estratégias de Controle e Validação

Como capturar o texto digitado e validar erros? Existem 4 formas principais, cada uma com seus Prós e Contras.

2.1. Usando [TextEditingController]

Criamos uma variável controladora e a vinculamos ao widget. É a forma mais "imperativa" de controlar o texto.

```
class _MyState extends State<MyWidget> {
  final _emailController = TextEditingController();

  @override
  void dispose() {
    _emailController.dispose(); // Obrigatório limpar a memória
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return TextFormField(
      controller: _emailController,
    );
  }

  void enviar() {
    print(_emailController.text); // Acessa o valor
    _emailController.clear(); // Limpa o campo
  }
}
```

- **Prós:**
  - Controle total sobre o texto (limpar, selecionar, substituir programaticamente).
  - Permite adicionar `listeners` para ouvir mudanças sem reconstruir a tela toda.
- **Contras:**
  - **Boilerplate:** Exige instanciar no `initState` (ou na declaração) e limpar no `dispose`.
  - A validação geralmente precisa ser feita manualmente com `if/else` se não estiver dentro de um `Form`.

2.2. Usando GlobalKey direto no Campo

Vinculamos uma chave única diretamente ao `TextFormField` para acessar seu estado interno.

```
final _fieldKey = GlobalKey<FormFieldState>();
```

```
TextFormField(
  key: _fieldKey,
  validator: (value) => value!.isEmpty ? 'Campo obrigatório' : null,
);

// Para validar apenas este campo específico:
_fieldKey.currentState?.validate();
```

- **Prós:**
  - Permite validar um único campo isoladamente (ex: validar um cupom de desconto sem validar o resto do formulário).
- **Contras:**
  - **Verborrágico:** Se tiver 10 campos, precisará de 10 chaves.
  - Dificulta a manutenção e a validação em lote ("Submit" do formulário).

2.3. Usando [Form] (A maneira "Padrão")

Envelopamos todos os inputs em um widget `Form` com uma única `GlobalKey`. É a arquitetura recomendada para cadastros.

```
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: Column(
    children: [
      TextFormField(
        validator: (v) => v!.isEmpty ? 'Erro' : null,
        onSaved: (v) => _usuario.nome = v,
      ),
      TextFormField(validator: (v) => ...),
      ElevatedButton(
        onPressed: () {
            // Valida TODOS os campos de uma vez
            if (_formKey.currentState!.validate()) {
                _formKey.currentState!.save(); // Chama o onSaved de todos
                // Enviar dados...
            }
        },
        child: Text('Enviar'),
      )
    ]
  )
)
```

- **Prós:**
  - **Centralização:** O método `.validate()` verifica todos os campos filhos de uma vez.
  - **UX:** Foca automaticamente no primeiro campo que estiver com erro e faz o scroll até ele.
  - Organizado e escalável para formulários grandes.

- **Contras:**
  - Exige um widget pai (`Form`) e a criação de uma `GlobalKey<FormState>`.

2.4. Usando [onChanged] com Estado (Reativo)

Atualizamos uma variável de estado a cada letra digitada utilizando o `setState` (ou Stores/Blocs/Controllers de gerência de estado).

```
String _email = '';

TextFormField(
  onChanged: (value) {
    setState(() {
      _email = value;
    });
  },
);

// O botão pode ser desabilitado reativamente
FilledButton(
  onPressed: _email.length > 5 ? enviar : null,
  child: Text('Enviar'),
)
```

- **Prós:**
  - **Reatividade:** Perfeito para habilitar/desabilitar o botão de "Enviar" em tempo real conforme o usuário digita.
  - Feedback visual imediato (ex: barra de força da senha mudando de cor enquanto digita).
- **Contras:**
  - **Performance:** O `setState` reconstruirá a tela a cada letra digitada. Em formulários gigantes, isso pode causar lentidão se não for bem gerenciado.
  - Acopla a regra de negócio diretamente na camada de visualização (UI).

---

3. Inputs de Seleção e Booleanos

3.1. [DropdownButtonFormField]

Input de seleção de lista (Select).

- Ideal para escolher uma opção dentro de uma lista fixa e limitada (ex: Estados, Gênero, Tipo de Usuário).
- Assim como o `TextFormField`, integra com o `Form` para validação e erros visuais.
- **Atenção:** O valor inicial (`value`) deve existir exatamente igual na lista de `items`, caso contrário o Flutter lançará um erro em tempo de execução.

3.2. [Checkbox] vs [CheckboxListTile]

- **Checkbox:** Apenas o quadradinho de seleção.

    - *Problema:* É pequeno e difícil de clicar em dispositivos móveis (touch target pequeno).
- **CheckboxListTile:** O quadradinho + Texto + Área de clique expandida.
    - **Uso:** "Aceito os termos de uso", "Lembrar minha senha".
    - O clique funciona tanto no quadrado quanto no texto, melhorando a acessibilidade.

3.3. [SwitchListTile]

Botão de "alavanca" (Toggle).

- **Diferença semântica:** Enquanto o Checkbox é para "aceite/seleção", o Switch é para **Estado/Ação** ou configuração.
- **Uso:** "Ativar notificações", "Modo Escuro", "Permitir localização".
- Geralmente denota que algo será ligado ou desligado no sistema imediatamente ou nas configurações do app.

Fontes

[Flutter-Docs]

- https://docs.flutter.dev/cookbook/forms

[Material-Widgets]

- https://api.flutter.dev/flutter/material/TextField-class.html (TextField)
- https://api.flutter.dev/flutter/material/TextFormField-class.html (TextFormField)
- https://api.flutter.dev/flutter/material/InputDecoration-class.html (InputDecoration)
- https://api.flutter.dev/flutter/widgets/Form-class.html (Form)
- https://api.flutter.dev/flutter/services/TextInputType-class.html (TextInputType)
- https://api.flutter.dev/flutter/services/TextInputAction-class.html (TextInputAction)
- https://api.flutter.dev/flutter/widgets/TextEditingController-class.html (TextEditingController)
- https://api.flutter.dev/flutter/material/DropdownButtonFormField-class.html (DropdownButtonFormField)
- https://api.flutter.dev/flutter/material/CheckboxListTile-class.html (CheckboxListTile)
- https://api.flutter.dev/flutter/material/SwitchListTile-class.html (SwitchListTile)

[Packages]

- https://pub.dev/packages/brasil_fields
- https://pub.dev/packages/mask_text_input_formatter